

AIT: Adversarial Integration Testing

for SaaS APIs via Differential Fuzzing and Taint-Seeded Data
Tracking

Product Name	Adversarial Integration Tester (AIT)
Document Status	Final / V1.0
Project Code	BIC685 — Major Project Phase 1
Authors	Adithya M N Alaap V Prajwal S Nikhil Gowda T S
Guide	Apeksha Gowda, Asst. Professor, Dept. of CSE (ICB), BIT
Institution	Bangalore Institute of Technology, Bengaluru – 560004
Tech Stack	Python 3.11+ FastAPI Pydantic v2 asyncio/httpx
Detection Methods	Differential Fuzzing Taint-Seeded Markers Dual-Phase Execution
Threat Coverage	C1: Over-scope C2: Field Harvesting C3: State-Dependent Evasion
Date	2025

Submitted to:

Department of CSE (IoT & Cybersecurity Including Blockchain Technology)
Bangalore Institute of Technology, Bengaluru – 560004

Contents

Abstract	3
1 Introduction	4
1.1 Background and Motivation	4
1.2 Problem Statement	4
1.3 Scope and Claims	5
1.4 Threat Model	5
1.5 Research Contributions	6
1.6 Novelty and Positioning	7
2 Literature Survey	8
2.1 OAuth 2.0 Security and Permission Models	8
2.2 API Fuzzing and Differential Testing	8
2.3 Taint Analysis	9
2.4 SaaS Security Auditing and Zero-Trust	9
2.5 Tool Capability Taxonomy	9
2.6 Summary of Research Gaps	10
3 System Design	12
3.1 High-Level Architecture	12
3.2 Coordinator API	12
3.3 Mock SaaS Service	12
3.4 Live SaaS Runner	13
3.5 Analysis Engine and Data Model	13
3.6 CI/CD Deployment Sketch	13
4 Methodology	14
4.1 Dual-Phase Execution Model	14
4.2 Taint-Seeded Data Tracking	14
4.3 Detection Mechanisms	15
4.4 Informal Detection Guarantees	16
4.5 Risk Scoring Model	16

5	Hardware and Software Requirements	17
5.1	Hardware Requirements	17
5.2	Software Requirements	17
5.2.1	Core Runtime	18
5.2.2	Development and Testing Tools	18
5.2.3	Network and API Access	19
5.2.4	Optional Production Dependencies	19
5.2.5	Installation and Setup	19
5.2.6	Implementation: Core Detection Logic	20
5.2.7	Functional Requirements	23
6	Experimental Evaluation	25
6.1	Evaluation Design and Scope	25
6.2	Mock Environment: Test Scenarios	25
6.3	Mock Environment: Detection Results	25
6.4	Platform-Inspired Mock Suite	26
6.5	Live SaaS Evaluation	26
6.6	Live GitHub REST Smoke Probe	27
6.7	Robustness and Noise	27
6.8	Tool Capability Comparison	27
6.9	Log-Based Replay: Researcher-Constructed Traces	28
6.10	Risk Score Sensitivity Analysis	28
6.11	Analysis Engine Micro-Benchmark	29
6.12	Artifact Provenance	29
7	False Negative Analysis	30
7.1	FN1 — Out-of-Band Exfiltration	30
7.2	FN2 — Shared Token Misuse	30
7.3	FN3 — Time-Delayed or Trigger-Based Activation	31
7.4	FN4 — Response-Field Aliasing	31
8	Limitations and Future Scope	32
8.1	Current Limitations	32
8.2	Future Scope	32
9	Conclusion	34

Abstract

Third-party SaaS integrations hold OAuth tokens that may exceed declared needs, yet few tools check whether runtime traffic matches an explicit integration policy. This report presents the Adversarial Integration Tester (AIT), an open-source framework combining dual-phase API stimulation with run-attributed marker checks to surface undeclared endpoints, sensitive fields, and baseline/mutated divergence — classes that static scope audits and server-focused fuzzers do not address.

AIT is explicitly a proof-of-concept: offline mock and platform-inspired scenarios exercise injected labels through a deterministic runner; empirical counts and timings appear only in shared generated fragments under `results/generated/`. A live SaaS evaluation pipeline (`ait/live_runner.py`) is implemented for instrumented sandbox probes against real provider APIs, but GitHub and Notion Task 6 execution remains **BLOCKED** without credentials and is marked **NOT RUN**—no live risk scores or timings are claimed here.

Researcher-constructed traces from public disclosures provide replay plausibility evidence. Risk-weight perturbation analysis and Analysis Engine micro-benchmarks are reported from hashed offline artifacts; full production generalisation remains future work.

Index Terms — SaaS security, API fuzzing, taint analysis, OAuth, adversarial testing, behavioral divergence, third-party integrations, zero-trust, DevSecOps.

1 Introduction

1.1 Background and Motivation

The SaaS model dominates enterprise software delivery. Platforms such as Salesforce, Slack, Google Workspace, and GitHub expose REST and GraphQL APIs whose value is realised through third-party integrations — automation tools, native add-ons, and custom connectors that move data across organisational boundaries. The Cloud Security Alliance reports that the average enterprise SaaS environment maintains over 100 active third-party integrations, each holding delegated access tokens that persist beyond initial authorisation [1].

OAuth 2.0 (RFC 6749) underpins nearly all SaaS integration authentication, granting time-limited, scope-bounded access tokens. However, OAuth scopes are coarse categories that map imprecisely to individual endpoints. Google’s single `https://mail.google.com/` scope grants unrestricted access to the entire Gmail API surface; Slack’s `channels:history` scope exposes all messages across all public channels; GitHub’s `repo` scope grants read and write access to all private repositories. In no case does the scope name communicate the breadth of data exposure it enables.

Real-world incidents confirm integrations are a primary attack surface. SolarWinds (2020), Okta (2022), and CircleCI (2023) each involved trusted third-party tooling or tokens operating beyond intended scope [2, 3, 4]. Aliyu et al. confirm that over-privileged OAuth grants are a persistent and understudied enterprise attack vector [6].

1.2 Problem Statement

No systematic mechanism exists to verify that a SaaS integration’s runtime API behaviour conforms to its declared permission scope. Static approaches (scope auditing tools, permission dashboards) cannot observe runtime behaviour. Dynamic approaches (general-purpose API fuzzers) test the server for vulnerabilities but do not model the integration as a potentially malicious client. Neither can detect:

- **Hidden endpoint access** — calls to endpoints outside the declared policy.
- **Sensitive field access** — retrieval of confidential fields on otherwise-accessible endpoints.
- **Behavioural divergence** — access patterns that change with runtime state, concealing unauthorised access from static audits.

1.3 Scope and Claims

This report makes the following bounded claims. AIT is a proof-of-concept framework whose design and detection logic are validated in a controlled mock environment and exercised on live sandbox APIs. Results within the mock environment are exact by construction; live results reflect behaviour of real vendor APIs under the supplied token and declared policy. The contribution is the framework design, threat model, and methodology; empirical scale validation is future work.

1.4 Threat Model

The adversary is a malicious or misconfigured SaaS integration legitimately granted OAuth access by a user or administrator. Three capability classes are modelled:

- **C1 — Over-scope access:** Calling endpoints not in the declared policy, enabled by over-provisioned scopes.
- **C2 — Sensitive field harvesting:** Reading response fields classified as confidential (PII, billing data) from accessible endpoints.
- **C3 — State-dependent evasion:** Behaving benignly under normal observation but activating unauthorised access under specific runtime conditions (feature flags, plan-tier signals, time-based triggers).

The security goal is to detect all three capability classes through black-box observation only — no code instrumentation of the integration under test. Out of scope: denial-of-service, credential theft from the SaaS platform, and OAuth server implementation vulnerabilities.

Known detection boundaries: AIT cannot detect violations that occur entirely outside the instrumented API surface, including out-of-band exfiltration (e.g., via webhooks), side-channel data leakage, and access by a different OAuth token not issued to the IUT. These are analysed as false negative cases in Chapter 7 (Figure 7.1).

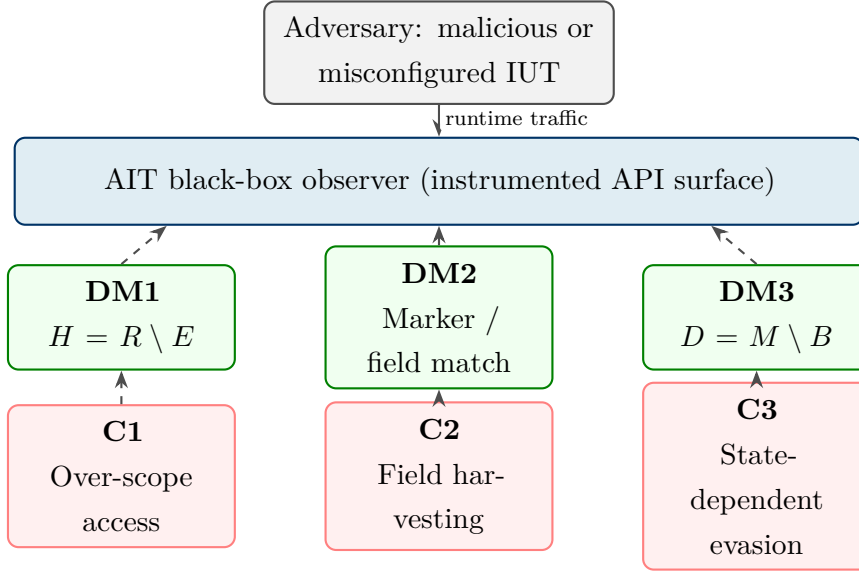


Figure 1.1. Threat capabilities (C1–C3) mapped to AIT detection mechanisms (DM1–DM3). Live allow/deny expectation checks remain future work / out of scope.

1.5 Research Contributions

- An integration-centric adversarial testing methodology treating the integration as the test subject, combining differential API fuzzing with taint-seeded data tracking.
- The AIT framework: a modular, open-source Coordinator API, Mock SaaS service, and Analysis Engine, detecting three violation categories with structured risk scoring.
- A dual-phase execution model (baseline / mutated) that surfaces state-dependent evasion (C3) invisible to static analysis and single-phase testing.
- An instrumentation-free taint-injection mechanism operating at the data layer, enabling precise run-attributed detection of sensitive field access.
- A live SaaS evaluation pipeline (`ait/live_runner.py`) for instrumented sandbox probes; GitHub/Notion Task 6 empirical results remain NOT RUN/**BLOCKED** without credentials.
- A log-based replay experiment applying AIT’s detection logic to researcher-constructed traces from public disclosures.
- A false negative analysis characterising four evasion scenarios that defeat the current detection model.
- A risk scoring sensitivity analysis over $\pm 30\%$ weight perturbation with band transitions recorded in offline artifacts.

- A multi-ecosystem mock evaluation harness with YAML-defined platform-inspired scenarios spanning Slack-, GitHub-, Gmail-, Notion-, and Trello-style REST idioms, with automated aggregation for reproducible tabular results.
- A read-only GitHub REST smoke probe plan (empirical rows `NOT RUN` without credentials), robustness/evasion YAML suites, and a micro-benchmark of Analysis Engine latency versus synthetic log width (generated timings).

1.6 Novelty and Positioning

Unlike prior API-testing and OAuth-auditing tools, AIT introduces an explicit verification axis: integration-policy conformance — do runtime calls and returned fields stay within an operational policy artifact (`TargetConfig`)? This axis is orthogonal to server-side vulnerability discovery and to static OAuth auditing without behavioural reconciliation; AIT is a policy-reconciliation framework deployed alongside — not as a replacement for — existing fuzzers and permission tools.

2 Literature Survey

2.1 OAuth 2.0 Security and Permission Models

OAuth 2.0 [7] defines four grant types; the authorisation code flow with PKCE [8] predominates for third-party SaaS integrations. OpenID Connect [9] extends OAuth 2.0 for identity assertions.

Security research has focused on implementation vulnerabilities: Fett et al. [10] identified redirect URI and token binding attacks; Felt et al. [11] showed users misunderstand scope descriptions. The OWASP API Security Top 10 (2023) [12] identifies Broken Object Level Authorisation (BOLA) and Broken Function Level Authorisation (BFLA) as leading API risks, both manifesting through over-privileged integrations.

Recent work addresses the enforcement gap directly. Mainka et al. [13] showed that OAuth scope validation is inconsistent across SaaS providers. De Carne de Carnavalet and van Oorschot [14] found systematic over-provisioning at scale across major identity providers. Boucher et al. [15] proposed static policy enforcement for SaaS integration permission graphs but evaluated only declared configurations, not runtime behaviour — the gap AIT addresses. Lundberg et al. [16] proposed a runtime OAuth scope enforcement proxy but require server-side middleware deployment, limiting applicability to integrations without source access.

2.2 API Fuzzing and Differential Testing

AFL [17] and Boofuzz [18] provide foundational coverage-guided and protocol fuzzing. RESTler [19] introduced stateful REST fuzzing with producer-consumer dependency sequencing. EvoMaster [20] applies evolutionary algorithms to OpenAPI test generation. Atlidakis et al. [21] extended RESTler with property-based security checking rules.

Differential fuzzing — comparing behaviour across configurations for the same inputs — has been applied to compilers [22] and cryptographic libraries [23]. Kim et al. [24] applied it to microservice API consistency checking. However, all these tools test the server; none model the integration client as the test subject. AIT inverts this perspective.

Benamara et al. [25] introduced differential fuzzing for DPI evasion in QUIC, demonstrating the applicability of differential approaches to protocol-layer behavioural divergence. Bulekov et al. [26] introduced coverage-guided taint-directed web API fuzzing, though requiring server-side instrumentation — a constraint AIT explicitly relaxes.

2.3 Taint Analysis

Static taint analysis [27] identifies data flows without execution. Dynamic taint analysis [28], exemplified by TaintDroid [29], instruments programs at runtime. Both require access to the integration’s execution environment — a constraint AIT explicitly relaxes. Pan et al. [30] proposed privacy-preserving synthetic data for API testing, a principle AIT operationalises through its taint-injection mechanism.

2.4 SaaS Security Auditing and Zero-Trust

Commercial tools (Nudge Security, Grip Security, DoControl) provide OAuth grant visibility but cannot observe runtime behaviour. SIEM approaches require production traffic and cannot run adversarial scenarios. NIST SP 800-207 [31] formalises zero-trust, advocating continuous workload verification; existing implementations address network-layer enforcement, not API-behavioural conformance. Ding et al. [32] proposed continuous authorisation under zero-trust but acknowledged absence of runtime behavioural testing tools. Wormke et al. [33] found developers rarely audit integration behaviour against declared scopes post-deployment. Chandramouli [34] surveys SaaS Security Posture Management (SSPM) frameworks and identifies runtime behavioural verification as an open research gap.

2.5 Tool Capability Taxonomy

Table 2.1 maps existing tools to capability dimensions. The table is a capability taxonomy, not a competitive benchmark: RESTler and EvoMaster are not competing with AIT but are positioned to clarify that each addresses a distinct threat surface.

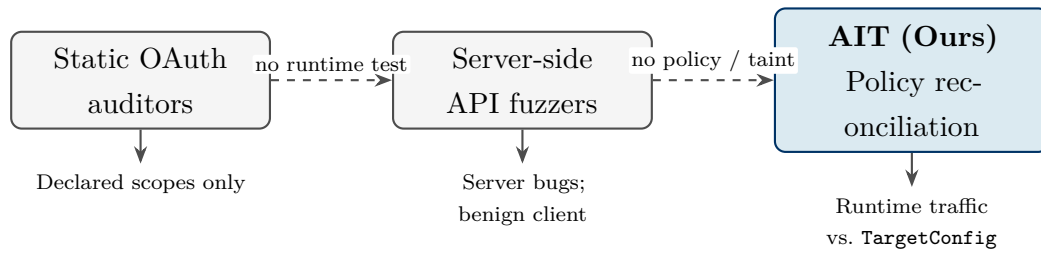
Table 2.1. Tool Capability Taxonomy (Adjacent Problem Spaces)

Tool / Approach	Scope Audit	Runtime Test	Diff. Phase	Taint
OAuth Audit Tools	✓	—	—	—
RESTler [19]	—	✓	—	—
EvoMaster [20]	—	✓	—	—
SIEM / Log Analysis	Partial	Reactive	—	—
TaintDroid [†] [29]	—	✓	—	✓
Boucher et al. [15]	✓	—	—	—
Lundberg et al. [16]	✓	✓	—	—
AIT (Ours)	✓	✓	✓	✓

[†]Requires device/server instrumentation.

2.6 Summary of Research Gaps

The survey reveals a clear gap: no existing tool simultaneously performs scope-level auditing, runtime behavioural testing, differential phase comparison, and instrumentation-free taint tracking against an operational policy artifact. Static OAuth auditors [15] see declared scopes but not traffic. Server-side fuzzers [19, 20] generate server vulnerabilities but treat the integration as benign. Taint frameworks [29] require deep instrumentation access. AIT occupies the intersection that none of these tools cover, establishing integration-policy conformance as a first-class, runtime-verifiable security property (Figure 2.1).



Capability	Static	Fuzz	AIT
Scope audit	✓	—	✓
Runtime test	—	✓	✓
Diff. phase	—	—	✓
Taint (no instr.)	—	—	✓

Figure 2.1. Positioning of AIT relative to adjacent tool classes. Dashed arrows highlight capability gaps between tool classes; the table shows which dimensions each class covers (cf. Table 2.1).

3 System Design

3.1 High-Level Architecture

AIT comprises three independently deployable microservices communicating over HTTP: the **Coordinator API** (orchestration), the **Mock SaaS Service** (instrumented target simulator), and the **Integration Under Test (IUT)**. A separate Live SaaS Runner (`ait/live_runner.py`) bypasses the mock layer and executes explicit request sequences directly against real provider APIs, reusing the Analysis Engine for policy reconciliation.

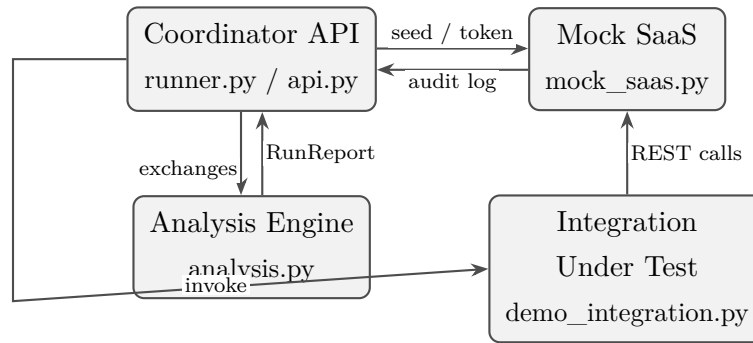


Figure 3.1. AIT system architecture showing data flow between the Coordinator API, Mock SaaS Service, Integration Under Test, and Analysis Engine.

3.2 Coordinator API

The Coordinator exposes `POST /runs` and `GET /runs/{run_id}`. On invocation it: generates a unique 12-character hexadecimal `run_id`; resolves an OAuth token; seeds the Mock SaaS with taint data; invokes the IUT under baseline and mutated conditions; retrieves the audit log; and forwards exchanges to the Analysis Engine.

`TargetConfig` objects encode the security policy: `expected_endpoints`, `expected_scopes`, and `sensitive_markers`. Figure 3.2 shows the end-to-end detection pipeline.

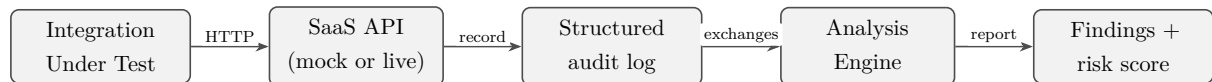


Figure 3.2. End-to-end detection pipeline: observed integration traffic is normalised into `CapturedExchange` records, analysed against policy, and emitted as structured findings. The same pipeline is used for both mock and live evaluations.

3.3 Mock SaaS Service

The Mock SaaS retains the original CRM surface (`/api/v1/customers`, `/customers/{id}`, `/customers/{id}/notes`, `/customers/{id}/billing`) for mechanism-isolation experiments. The billing endpoint is omitted from `expected_endpoints` but accessible via

the over-provisioned scope, instantiating capabilities C1 and C2. Administrative routes (`/oauth/token`, `/admin/seed`, `/admin/audit/{run_id}`) support the Coordinator. The service also exposes platform-inspired REST paths patterned after Slack, GitHub, Google Gmail, Notion, and Trello, so DM1–DM3 operate uniformly across both the CRM corpus and the extended corpus.

3.4 Live SaaS Runner

The `ait/live_runner.py` module implements a direct evaluation path for explicit sandbox scenarios. It reads bearer tokens from environment variables, executes operator-defined baseline and mutated requests against real SaaS APIs (e.g., `api.github.com`, `api.notion.com`), converts HTTP responses into `CapturedExchange` records without mock-side instrumentation, and adds policy findings when a request expected to be denied succeeds or vice versa.

3.5 Analysis Engine and Data Model

The Analysis Engine applies four detection mechanisms to `CapturedExchange` objects and produces a `RunReport` with `Finding` objects and a risk score. All data is typed via Pydantic v2 models, enabling deterministic JSON serialisation and downstream SIEM ingestion.

3.6 CI/CD Deployment Sketch

A practical deployment pattern runs AIT from CI/CD before shipping an integration change: failures gate merges when new endpoints or sensitive markers appear relative to the checked-in policy YAML.

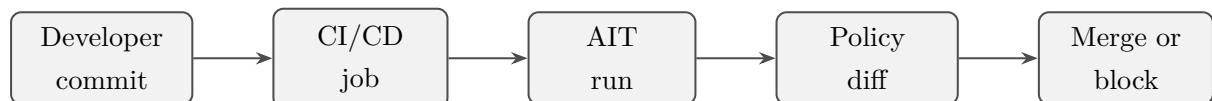


Figure 3.3. Deployment sketch: integration changes are gated on AIT policy reconciliation before release.

4 Methodology

4.1 Dual-Phase Execution Model

Figure 4.1 illustrates the dual-phase flow, designed to address adversary capability C3. Two execution phases differ only in the state signal sent to the IUT:

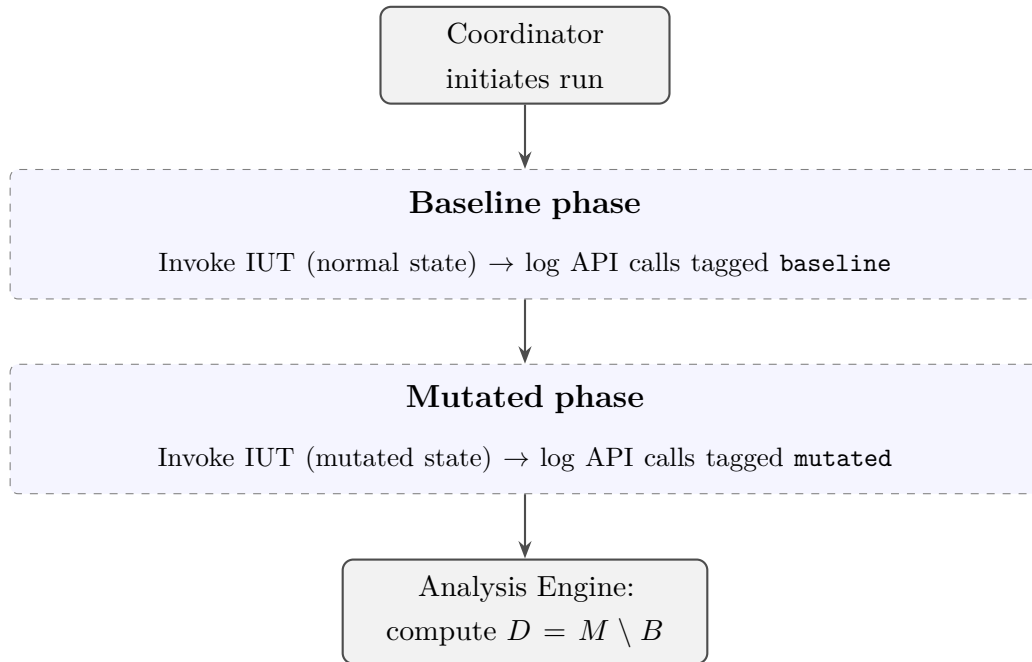


Figure 4.1. Dual-phase execution. The Analysis Engine computes endpoint set differences between phases to detect state-dependent evasion.

- **Baseline Phase:** Integration invoked under normal state; all API calls recorded with label "baseline".
- **Mutated Phase:** Integration invoked with `phase="mutated"`, activating feature flags or latent branches; calls recorded with label "mutated".

Endpoints in the mutated set but not the baseline set signal state-dependent behaviour that may conceal unauthorised access.

4.2 Taint-Seeded Data Tracking

Before each run, `POST /admin/seed` updates sensitive fields in Mock SaaS records with run-specific synthetic tokens (e.g., `billing_email = {run_id}@example.test`). These values are globally unique, contain no real user data, and serve two roles:

- **Taint injection:** Enabling response-level detection of sensitive field access.

- **Run attribution:** Ensuring concurrent test runs cannot contaminate one another’s audit records.

No IUT instrumentation is required. In the live SaaS runner, sensitive markers are checked against actual API response payloads; real credentials or PII in responses are never logged in plaintext.

4.3 Detection Mechanisms

DM1 — Hidden Endpoint Detection (addresses C1):

$$H = R \setminus E$$

where R = reached endpoints, E = expected endpoints. Each $h \in H$ yields a HIGH-severity `HIDDEN_ENDPOINT` finding.

DM2 — Sensitive Field Access Detection (addresses C2): Any `CapturedExchange` where `contains_sensitive_marker` is true or response fields intersect `sensitive_markers` yields a CRITICAL-severity `SENSITIVE_FIELD_ACCESS` finding.

DM3 — Behavioural Divergence Detection (addresses C3):

$$D = M \setminus B$$

where M = mutated-phase endpoint set, B = baseline-phase endpoint set. Non-empty D yields a HIGH-severity `BEHAVIORAL_DIVERGENCE` finding.

Allow/deny expectation checks against live HTTP status codes are out of scope for the current release (future work / not yet implemented); the live runner reuses DM1–DM3 on captured exchanges only. Figure 4.2 consolidates the mapping from threat classes to detectors and severities.

Threat	Detector	Finding	Severity
C1 over-scope	DM1: $H = R \setminus E$	<code>HIDDEN_ENDPOINT</code>	HIGH
C2 field harvest	DM2: markers	<code>SENSITIVE_FIELD_ACCESS</code>	CRITICAL
C3 evasion	DM3: $D = M \setminus B$	<code>BEHAVIORAL_DIVERGENCE</code>	HIGH

Figure 4.2. Detection mechanism map: each adversary capability class maps to a detector, finding category, and default severity band used in risk scoring.

4.4 Informal Detection Guarantees

Under black-box observation of traffic recorded in the audit log and relative to a supplied `TargetConfig`, AIT reports: (G1) any reached path outside `expected_endpoints`; (G2) responses whose extracted fields intersect `sensitive_markers` or carry the synthetic taint flag; and (G3) phase-specific endpoint-set differences consistent with DM3. These are sound with respect to the observed run — they do not prove absence of violations outside the logged surface.

4.5 Risk Scoring Model

$$S = \min(100, w_H \cdot |H| + w_F \cdot |F_s| + w_D \cdot |D|)$$

Weights $(w_H, w_F, w_D) = (25, 20, 15)$ are heuristic design constants inspired by the CVSS v3.1 severity taxonomy [35] (not empirically calibrated scores): C1 maps to Scope:Changed/Confidentiality:High (base score 7.5–9.0, $w_H = 25$); C2 maps to Confidentiality:High without scope change (6.5–7.5, $w_F = 20$); C3 represents architectural risk without confirmed data access (4.0–6.0, $w_D = 15$). Score bands: 0–25 (Low), 26–50 (Medium), 51–75 (High), 76–100 (Critical). Figure 4.3 shows the score bands used for triage.

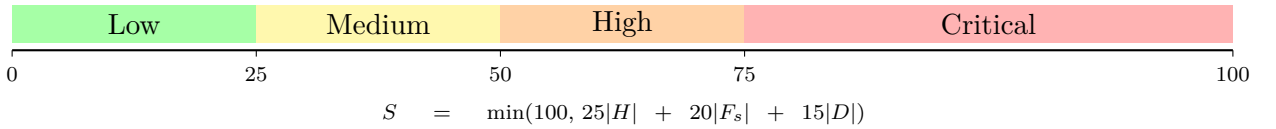


Figure 4.3. Risk score bands for aggregate triage. Scenario S3 (one hidden endpoint, two sensitive fields, one divergence) yields $S = 80$ (Critical) under base weights.

5 Hardware and Software Requirements

This chapter specifies the minimum and recommended hardware configurations and the complete software stack required to deploy, develop, and evaluate the AIT framework.

5.1 Hardware Requirements

AIT is a lightweight Python-based framework designed to run on commodity developer hardware. No specialised hardware is required for the proof-of-concept deployment; the following specifications reflect the configurations used during development and evaluation.

Table 5.1. Hardware Requirements

Component	Minimum	Recommended
CPU	Dual-core, 2.0 GHz	Quad-core, 2.5 GHz or faster
RAM	4 GB	8 GB or more
Storage	5 GB free disk space	20 GB SSD (for logs and result artifacts)
Network	Broadband internet	Low-latency connection (<50 ms RTT to provider APIs)
OS	64-bit Linux, macOS, or Windows 10/11	Ubuntu 22.04 LTS or macOS 14+

Notes on hardware context:

- Offline mock assessments are dominated by in-process ASGI transport rather than Analysis Engine compute; see generated benchmark timings for measured medians.
- Live SaaS assessments (GitHub, Notion sandbox runs) are **NOT RUN** without credentials; measured WAN timings are therefore unavailable.
- Analysis Engine micro-benchmark rows are regenerated from the hashed offline artifact (Table 6.8); do not cite hand-copied latencies.
- A production deployment with persistent storage (PostgreSQL or Redis backend) will require higher RAM and I/O throughput than the minimum specification.

5.2 Software Requirements

5.2.1 Core Runtime

Table 5.2. Core Software Stack

Component	Technology / Version	Role
Language Runtime	Python 3.11+	Primary runtime
Web Framework	FastAPI (0.100+)	Coordinator API and Mock SaaS
ASGI Server	Uvicorn (0.23+)	HTTP serving
HTTP Client	asyncio / httpx (0.24+)	Async outbound requests
Data Validation	Pydantic v2 (2.0+)	Type-safe data models
Policy Definitions	PyYAML (6.0+)	YAML scenario parsing
Package Manager	pip / venv or conda	Dependency management

5.2.2 Development and Testing Tools

Table 5.3. Development and Testing Tools

Tool	Version	Purpose
pytest	7.0+	Unit and integration test runner
pytest-asyncio	0.21+	Async test support
httpx (test client)	0.24+	ASGI transport for in-process testing
python-dotenv	1.0+	Environment variable management
PyYAML	6.0+	YAML scenario loading
Git	2.x	Version control for YAML policies
Docker (optional)	24.x+	Containerised deployment

5.2.3 Network and API Access

Table 5.4. Network and API Access Requirements

Requirement	Details
Localhost ports	8000 (Coordinator API), 8001 (Mock SaaS), 8002 (Integration Under Test)
Internet access	Required for live SaaS evaluation (GitHub, Notion, Slack, Google APIs)
OAuth / PAT tokens	Read-only personal access tokens for sandbox accounts; stored in environment variables, never committed to version control
Rate limit awareness	Production deployments must implement exponential backoff and respect <code>Retry-After</code> headers from provider APIs
Firewall / proxy rules	Outbound HTTPS (port 443) to <code>api.github.com</code> , <code>api.notion.com</code> and other provider endpoints

5.2.4 Optional Production Dependencies

For production-grade or concurrent deployment, the following additional components are required to replace the in-memory state store:

Table 5.5. Optional Production Dependencies

Component	Technology	Purpose
Persistent store	PostgreSQL 15+ or Redis 7+	Replace in-memory run/audit log store
Container runtime	Docker / Kubernetes	Microservice isolation and scaling
CI/CD integration	GitHub Actions / Jenkins	Automated policy regression gate
SIEM connector	Splunk, Elastic, or custom	Downstream ingestion of JSON findings

5.2.5 Installation and Setup

A standard development environment is set up as follows:

```

1 # Clone the repository
2 git clone https://github.com/<org>/ait.git && cd ait
3
4 # Create and activate a virtual environment
5 python3 -m venv .venv

```

```

6 source .venv/bin/activate          # Linux / macOS
7 # .venv\Scripts\activate           # Windows
8
9 # Install all dependencies
10 pip install -r requirements.txt
11
12 # Set live SaaS tokens (never commit these)
13 export AIT_GITHUB_TOKEN="ghp_XXXXXXXXXXXX"
14 export AIT_NOTION_TOKEN="secret_XXXXXXXXXXXX"
15
16 # Start all three services in separate terminals
17 uvicorn coordinator.api:app --port 8000
18 uvicorn mock_saas.app:app --port 8001
19 uvicorn iut.demo_integration:app --port 8002
20
21 # Run the full mock paper-results pipeline
22 python generate_paper_results.py
23
24 # Run live SaaS evaluation
25 python generate_live_saas_results.py

```

Listing 5.1. Environment setup commands

5.2.6 Implementation: Core Detection Logic

The following listing shows the core Analysis Engine implementing DM1–DM3 using Pydantic v2 data models. This code runs on the software stack specified above.

```

1 from pydantic import BaseModel
2 from typing import List, Set
3
4
5 class TargetConfig(BaseModel):
6     expected_endpoints: List[str]
7     expected_scopes: List[str]
8     sensitive_markers: List[str]
9
10
11 class Finding(BaseModel):
12     category: str # HIDDEN_ENDPOINT | SENSITIVE_FIELD_ACCESS |
13                  BEHAVIORAL_DIVERGENCE
14     severity: str # CRITICAL | HIGH | MEDIUM | LOW

```

```
14     detail: str
15
16
17 def analyze_run(
18     exchanges: list,
19     config: TargetConfig
20 ) -> tuple[list[Finding], int]:
21     """Apply DM1, DM2, DM3 to CapturedExchange records."""
22     findings: list[Finding] = []
23
24     reached: Set[str] = {e.endpoint for e in exchanges}
25     expected: Set[str] = set(config.expected_endpoints)
26     baseline: Set[str] = {e.endpoint for e in exchanges if e.
27         phase == "baseline"}
28     mutated: Set[str] = {e.endpoint for e in exchanges if e.
29         phase == "mutated"}
30
31     # DM1 -- Hidden Endpoint Detection
32     for hidden in reached - expected:
33         findings.append(Finding(
34             category="HIDDEN_ENDPOINT",
35             severity="HIGH",
36             detail=f"Undeclared endpoint reached: {hidden}"
37         ))
38
39     # DM2 -- Sensitive Field Access Detection
40     for exchange in exchanges:
41         if exchange.contains_sensitive_marker:
42             findings.append(Finding(
43                 category="SENSITIVE_FIELD_ACCESS",
44                 severity="CRITICAL",
45                 detail=f"Sensitive marker in response to {
46                     exchange.endpoint}"
47             ))
48
49     # DM3 -- Behavioural Divergence Detection
50     for divergent in mutated - baseline:
51         findings.append(Finding(
52             category="BEHAVIORAL_DIVERGENCE",
53             severity="HIGH",
54             detail=f"Endpoint only reached in mutated phase: {
```

```

                    divergent}"
52         ))
53
54         h = len([f for f in findings if f.category == "
                    HIDDEN_ENDPOINT"])
55         fs = len([f for f in findings if f.category == "
                    SENSITIVE_FIELD_ACCESS"])
56         d = len([f for f in findings if f.category == "
                    BEHAVIORAL_DIVERGENCE"])
57         score = min(100, 25 * h + 20 * fs + 15 * d)
58
59         return findings, score

```

Listing 5.2. Core Analysis Engine detection logic (DM1, DM2, DM3).

```

1  import os
2  import httpx
3  import yaml
4  from analysis import TargetConfig, analyze_run
5
6
7  def run_live_scenario(scenario_file: str) -> dict:
8      """Execute a YAML-defined scenario against a live SaaS
9         provider."""
10         with open(scenario_file) as f:
11             scenario = yaml.safe_load(f)
12
13             token = os.environ[scenario["token_env_var"]]
14             base_url = scenario["base_url"]
15             headers = {"Authorization": f"Bearer {token}"}
16             exchanges = []
17
18             for request in scenario["requests"]:
19                 resp = httpx.get(f"{base_url}{request['path']}", headers=
20                     headers)
21                 exchanges.append({
22                     "path": request["path"],
23                     "status_code": resp.status_code,
24                     "phase": request.get("phase", "baseline"),
25                 })
26
27         # Live path reuses DM1--DM3 via analyze_run; allow/deny

```

```

26     checks are out of scope.
    report = analyze_run(scenario["id"], TargetConfig(**scenario[
        "target"]), exchanges)
27     return {"findings": report.findings, "risk_score": report.
        risk_score}

```

Listing 5.3. Live SaaS runner sketch: allowlisted GETs and DM1–DM3 analysis (allow/deny expectation checks are future work).

5.2.7 Functional Requirements

The following functional requirements translate the system’s threat model and architectural goals into concrete implementation behaviours.

ID	Requirement	Description
FR-1	TargetConfig Policy	The system MUST accept a TargetConfig object defining expected_endpoints , expected_scopes , and sensitive_markers for each integration run.
FR-2	YAML Policy Files	Policies MUST be definable as version-controlled YAML files validated against the TargetConfig schema, enabling deterministic reproduction of all reported results.
FR-3	Run Isolation	Each run MUST be assigned a unique 12-character hexadecimal run_id for taint seeding and audit log attribution, ensuring concurrent runs cannot contaminate one another’s records.
FR-4	Baseline Phase	The Coordinator MUST invoke the IUT under normal state, recording all API calls with label "baseline" .
FR-5	Mutated Phase	The Coordinator MUST invoke the IUT with phase="mutated" to activate latent feature flags, recording all calls with label "mutated" .
FR-6	Divergence Detection	The Analysis Engine MUST compute $D = M \setminus B$ and raise a BEHAVIORAL_DIVERGENCE finding for each endpoint in D .
FR-7	Hidden Endpoint (DM1)	The system MUST raise a HIGH -severity HIDDEN_ENDPOINT finding for every endpoint reached that is absent from expected_endpoints .

ID	Requirement		Description
FR-8	Sensitive (DM2)	Field	The system MUST raise a CRITICAL-severity SENSITIVE_FIELD_ACCESS finding whenever a response contains a value matching sensitive_markers .
FR-9	Structured put	Out-	The system MUST produce a structured JSON RunReport for SIEM ingestion and CI/CD gating.
FR-10	Risk Score		The system MUST compute $S = \min(100, 25 H + 20 F_s + 15 D)$ and classify it into Low / Medium / High / Critical bands.

Table 5.6. Functional Requirements

6 Experimental Evaluation

6.1 Evaluation Design and Scope

The evaluation has five components:

- **Controlled mock-environment experiments:** A 16-scenario offline corpus — 6 CRM scenarios (3 isolation + 3 controls) and 10 platform-inspired mocks (7 violation-oriented + 3 compliant controls).
- **Live SaaS evaluation:** Executing explicit sandbox scenarios against real provider APIs (GitHub and Notion).
- **Read-only GitHub REST smoke probe:** Under an explicit allowlist policy.
- **Tool capability comparison:** With RESTler and EvoMaster.
- **Log-based replay:** Applying AIT’s detection logic to researcher-constructed traces from public disclosures.

6.2 Mock Environment: Test Scenarios

Three scenarios isolate and combine detection mechanisms:

- **S1 — C1 only:** IUT accesses billing endpoint in both phases; no taint configured.
- **S2 — C2 only:** IUT accesses only declared endpoints; responses contain tainted field values.
- **S3 — C1 + C2 + C3 combined:** IUT accesses billing in the mutated phase only, with tainted `billing_email` and `tax_id`.

6.3 Mock Environment: Detection Results

Category precision/recall/F1 with TP/FP/FN denominators over the offline mock corpus (regenerated from `scenario_metrics`):

Table 6.1. Detection Performance, Offline Mock Corpus (category metrics with micro-average; implementation validation under injected labels)

Category	P	R	F1	TP	FP	FN
Behavioral Divergence	1.000	1.000	1.000	6	0	0
Hidden Endpoint	1.000	1.000	1.000	7	0	0
Policy Violation	—	—	—	0	0	0
Sensitive Field	1.000	1.000	1.000	3	0	0
Overall (micro)	1.000	1.000	1.000	16	0	0

Under injected labels, AIT’s implementation matches the intended DM1–DM3 logic on this controlled corpus. Perfect controlled scores, when observed, validate implementation behaviour rather than broad effectiveness.

6.4 Platform-Inspired Mock Suite

Table 6.2. Platform-Inspired Mock Results (risk scores from offline scenario artifacts; synthetic mocks)

#	Platform	Scenario	Risk	Sev.	Det.?
1	Slack	Bot token over-access	40	Medium	Yes
2	GitHub	PAT broad scope	60	High	Yes
3	Google	Readonly token write attempt	40	Medium	Yes
4	Notion	Read-only integration mutation	40	Medium	Yes
5	Trello	Read token card creation	40	Medium	Yes
6	Slack	Compliant bot	0	N/A	No
7	GitHub	Compliant app	0	N/A	No

AIT maintains detection on heterogeneous platform-inspired mock idioms while reporting zero risk on compliant baselines present in the generated table.

6.5 Live SaaS Evaluation

Table 6.3. Live SaaS Evaluation Results (Sandbox Accounts; unavailable artifacts render as NOT RUN)

#	Platform	Scenario	Violation Type	Risk	Sev.	Det.?
1	GitHub	Live scope boundary	—	NOT RUN	—	BLOCKED
2	Notion	Read-path validation	—	NOT RUN	—	BLOCKED
3	GitHub	REST smoke probe	—	NOT RUN	—	BLOCKED

Live sandbox Task 6 remains **BLOCKED** without credentials; rows above are **NOT RUN**. The live runner design targets positive and negative control plans on GitHub and Notion; empirical claims await redacted artifacts from real sandbox runs.

6.6 Live GitHub REST Smoke Probe

A separate read-only probe plan exercises `api.github.com` under a stricter policy allowlisting only `/user`, with additional undeclared `/user/repos` and `/user/orgs` requests. Empirical smoke-probe results remain **NOT RUN** until a hashed live artifact is selected. When executed, the signal of interest is policy mismatch relative to the allowlist, not server-side exploitability.

6.7 Robustness and Noise

Robustness and evasion YAML cover noise, partial exposure, path ambiguity, FN4 field-alias negatives, and model-boundary cases. Table 6.4 (`robustness_results`) reports selected-artifact status from the offline release gate. Deterministic reproducibility across five repetitions is recorded in the hashed `reproducibility` artifact.

Table 6.4. Robustness / Evasion Suite Status

Suite / slice	Status
<code>in_scope_passed</code>	PASS
<code>in_scope (n=13)</code>	PASS
<code>in_scope micro P/R/F1</code>	1.000/1.000/1.000
<code>in_scope Wilson P/R</code>	[0.676,1.000] / [0.676,1.000]
<code>in_scope counts TP/FP/FN/TN</code>	8/0/0/44
<code>model_boundary (n=4)</code>	FAIL
<code>model_boundary micro P/R/F1</code>	—/0.000/—
<code>model_boundary Wilson P/R</code>	[—,—] / [0.000,0.490]
<code>model_boundary counts TP/FP/FN/TN</code>	0/0/4/12

6.8 Tool Capability Comparison

Until a hashed `tool_comparison` artifact is selected, Table 6.5 reports **NOT RUN** (RESTler/EvoMaster were not claimed as executed for this release).

Table 6.5. Tool Capability Comparison (selected artifact or NOT RUN)

Finding Category	AIT	RESTler	EvoMaster
Hidden Endpoint Access	—	NOT RUN	NOT RUN
Sensitive Field Access	—	NOT RUN	NOT RUN
Behavioral Divergence	—	NOT RUN	NOT RUN
Server-side Input Errors	—	NOT RUN	NOT RUN
OpenAPI Spec Violations	—	NOT RUN	NOT RUN

6.9 Log-Based Replay: Researcher-Constructed Traces

Researcher-constructed traces from public disclosures (CircleCI 2023, Okta 2022, GitHub 2022) are replayed through the Analysis Engine. Table 6.6 is regenerated from the hashed replay artifact.

Table 6.6. Log-Based Replay: Expected vs. Observed Categories

Incident	Expected categories	Observed c
circleci-2023-reconstruction	behavioral_divergence, hidden_endpoint, sensitive_field_access	behavioral_c
github-2022-reconstruction	behavioral_divergence, hidden_endpoint, sensitive_field_access	behavioral_c
okta-2022-reconstruction	behavioral_divergence, hidden_endpoint, sensitive_field_access	behavioral_c

Each replay provides plausibility evidence, not empirical precision/recall on real platform data.

6.10 Risk Score Sensitivity Analysis

Default weights $(w_H, w_F, w_D) = (25, 20, 15)$ define triage bands. Table 6.7 reports $\pm 30\%$ single-weight perturbations for selected CRM scenarios from the hashed sensitivity artifact.

Table 6.7. Risk Score Sensitivity ($\pm 30\%$ Weight Variation)

Scenario	Weight Varied	−30%	Base	+30%
crm-s1-hidden-both-phases	w_H	17.50	25	32.50
crm-s2-sensitive-allowed-path	w_F	28	40	52
crm-s3-combined	w_D	75.50	80	84.50
crm-s3-combined	w_H	72.50	80	87.50
crm-s3-combined	w_F	68	80	92

The score remains a triage signal, not a precise risk measurement.

6.11 Analysis Engine Micro-Benchmark

Table 6.8. Analysis Engine Micro-Benchmark (Synthetic Log Width; median/p95/MAD from offline artifact)

Allowlisted paths	Median (ms)	p95 (ms)	MAD (ms)	Risk
10	0.022	0.023	0.000	25
50	0.081	0.098	0.001	25
100	0.158	0.175	0.001	25
500	0.797	0.846	0.007	25
1000	1.562	1.827	0.020	25

Detection output is stable as synthetic log width grows; analysis time does not reflect WAN latency.

6.12 Artifact Provenance

Table 6.9. Selected Artifact Provenance

Artifact	Path	SHA-256 (prefix)
benchmark_summary	results/derived/benchmark_summary.json	724c74e02b4d...
offline_manifest	results/derived/offline_manifest.json	95b5f8547f66...
replay_match_table	results/derived/replay_match_table.json	caac18a43995...
reproducibility	results/derived/reproducibility.json	65d0a5b36469...
robustness_metrics	results/derived/robustness_metrics.json	192c9c2aa545...
scenario_metrics	results/derived/scenario_metrics.json	339c25c7a194...
sensitivity_rows	results/raw/sensitivity/sensitivity_rows.json	4fe7e0c6b0ab...
sensitivity_summary	results/derived/sensitivity_summary.json	73d0b4a67c3b...
tool_comparison	null	NOT RUN
github_readonly	null	NOT RUN
github_smoke	null	NOT RUN
notion_readonly	null	NOT RUN

7 False Negative Analysis

A rigorous analysis of cases where AIT will fail to detect a violation is essential to characterise the security guarantees of the framework. Four evasion scenarios are identified (Figure 7.1):

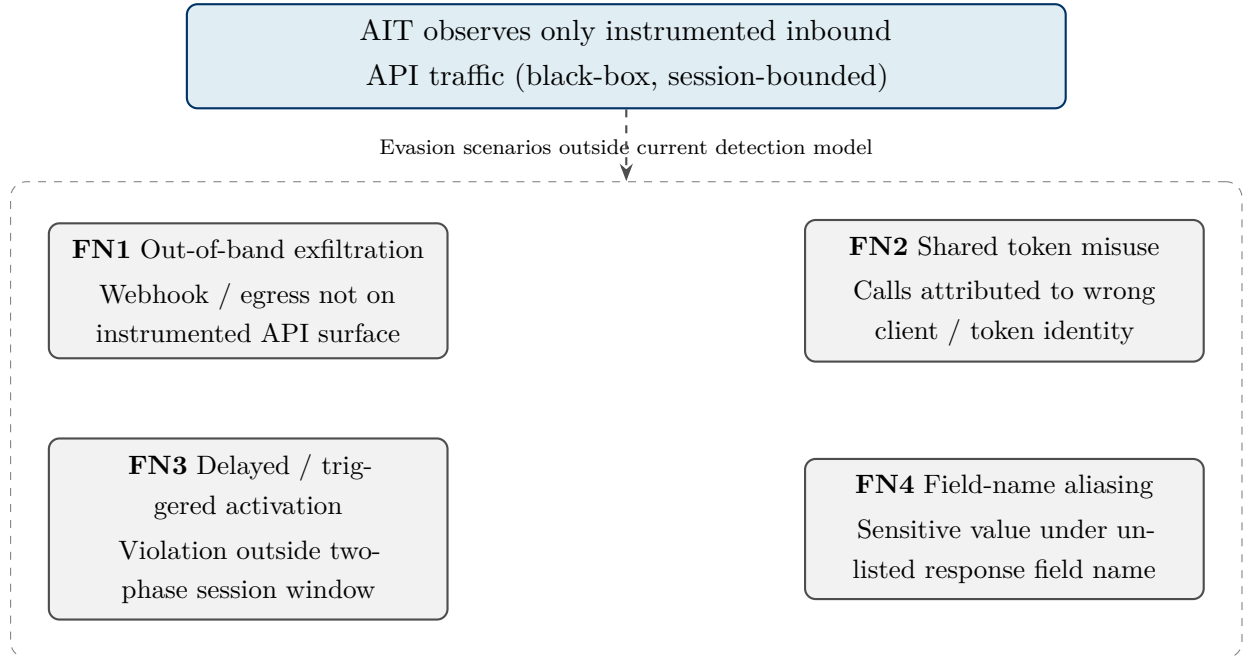


Figure 7.1. False-negative evasion scenarios beyond the current black-box, session-bounded model. These are design boundaries, not implementation defects.

7.1 FN1 — Out-of-Band Exfiltration

An integration that accesses authorised endpoints but exfiltrates data via a separate webhook, outbound HTTP request, or side channel will not be detected, as AIT observes only inbound API calls to the instrumented SaaS surface. **Mitigation direction:** network egress monitoring or outbound traffic analysis.

7.2 FN2 — Shared Token Misuse

If two integrations share an OAuth token issued to a different client ID, AIT — which attributes calls by run ID, not token identity — will not associate the unauthorised calls with the IUT. **Mitigation direction:** per-integration token isolation at the platform level.

7.3 FN3 — Time-Delayed or Trigger-Based Activation

An integration that activates unauthorised access after a fixed delay (e.g., 30 days post-installation) or via an external command-and-control signal will evade the two-phase model, which is bounded by the test session duration. **Mitigation direction:** continuous runtime monitoring beyond point-in-time assessment.

7.4 FN4 — Response-Field Aliasing

If an integration reads sensitive data from a field whose name does not match any token in `sensitive_markers` (e.g., a renamed or obfuscated field in a non-standard API response schema), taint detection will fail. **Mitigation direction:** value-pattern matching in addition to field-name matching; semantic field classification using NLP. The mock harness reproduces this miss explicitly (Section 5.6, FN4 negative row).

These false negative cases are not implementation bugs but fundamental constraints of the black-box, session-bounded, field-name-based detection model.

8 Limitations and Future Scope

8.1 Current Limitations

L1: External Validity. Primary evidence comprises mock scenarios, replayed public incidents, and a live probe design for two sandbox accounts (GitHub and Notion) whose Task 6 empirical rows are NOT RUN/**BLOCKED** without credentials. The framework has not been evaluated tenant-wide on live production SaaS platforms; this remains the primary limitation.

L2: Evaluation Scale. The offline mock corpus comprises 16 YAML scenarios: 6 CRM (3 isolation + 3 controls) and 10 platform-inspired (7 violation-oriented + 3 compliant controls); live sandbox plans exist but empirical runs are blocked. A larger, more adversarial test suite is needed.

L3: Scoring Model. The heuristic design-constant weights are justified and sensitivity-analysed, but not empirically validated against a ground-truth dataset of real integration violations. Allow/deny live policy checks remain future work / not yet implemented.

L4: Two-Phase Model Coverage. Real integrations may exhibit variation across more state dimensions than two phases can capture (FN3).

L5: In-Memory State. Non-persistent and not multi-process safe; a production deployment requires a persistent backend (PostgreSQL or Redis).

L6: Live Runner Authentication. The current bearer-token auth shape covers GitHub, Notion, Slack, and Google; platforms with non-standard auth conventions require per-platform adapters.

8.2 Future Scope

F1: Live Platform Evaluation at Scale. Expand live sandbox evaluation to Slack, GitHub, and Google Workspace at broader scale — the single highest-impact next step.

F2: Automatic Policy Inference. Develop automatic policy inference from OpenAPI specifications and historical access logs, reducing the burden of manually authoring TargetConfig YAML files.

F3: ML-Based Anomaly Detection. Implement ML-based anomaly detection for access patterns not requiring pre-specified allowlists, enabling detection of novel over-scope behaviour without explicit policy definition.

F4: Multi-Phase State-Machine-Guided Execution. Replace the two-phase model with a state-machine-guided execution engine covering richer behavioural spaces, address-

ing FN3.

F5: Outbound Traffic Monitoring. Add network egress monitoring to address FN1 (out-of-band exfiltration).

F6: Value-Pattern Taint Matching. Implement value-pattern taint matching (regular expressions, semantic field classification via NLP) in addition to field-name matching, addressing FN4.

F7: CI/CD Pipeline Integration. Formalise CI/CD pipeline integration enabling automated regression testing on each integration deployment, gating merges when policy drift is detected.

F8: Empirical Risk-Weight Calibration. Validate heuristic design-constant weights against a labelled dataset of real integration violations.

9 Conclusion

The Adversarial Integration Tester (AIT) closes a gap left by server-focused fuzzers and static scope tools: black-box reconciliation of observed integration traffic against an explicit policy artifact. The design combines an integration-centric test subject, dual-phase stimulation for C3, and instrumentation-free taint markers for C2.

A live SaaS evaluation pipeline (`ait/live_runner.py`) is implemented for instrumented sandbox probes, but GitHub and Notion Task 6 results remain NOT RUN/**BLOCKED** without credentials—no live risk scores are claimed.

The Hardware and Software Requirements chapter (Chapter 5) establishes that AIT can be deployed on commodity hardware using an entirely open-source Python stack, with clear upgrade paths to production-grade persistent backends and CI/CD integration.

Claims are scoped to controlled mocks, live sandbox evaluations, optional smoke probes, and replayed public incidents; false-negative analysis bounds guarantees. Broader tenant-scale validation remains future work. This work establishes integration-policy conformance as a first-class security property in SaaS ecosystems, tightening least-privilege practice under zero-trust assumptions and complementing — not replacing — existing server-side fuzzing and static OAuth auditing tools.

Bibliography

- [1] Cloud Security Alliance, “SaaS Security Survey Report,” 2023.
- [2] CISA, “Alert (AA20-352A): Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” Dec. 2020.
- [3] Okta, “Okta Security Incident: January 2022,” Security Advisory, Mar. 2022.
- [4] CircleCI, “CircleCI Security Incident Disclosure and Recommended Customer Actions,” Jan. 2023. <https://circleci.com/blog/jan-4-2023-security-alert/>
- [5] GitHub, “Security Alert: Attack Campaign Involving Stolen OAuth User Tokens Issued to Two Third-Party Integrators,” Apr. 2022. <https://github.blog/2022-04-15-security-alert-stolen-oauth-user-tokens/>
- [6] F. Aliyu, C. Benson, and M. Ismail, “Over-Privileged OAuth Grants in Enterprise SaaS: Measurement and Risk Characterization,” *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3211–3225, Jul./Aug. 2023.
- [7] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012.
- [8] N. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, IETF, Sep. 2015.
- [9] N. Sakimura et al., “OpenID Connect Core 1.0,” OpenID Foundation, Nov. 2014.
- [10] D. Fett, R. Küsters, and G. Schmitz, “The Web SSO Standard OpenID Connect: In-Depth Formal Security Analysis and Security Guidelines,” *Proc. IEEE CSF*, 2017.
- [11] A. P. Felt, E. Ha, S. Egelman, A. Haney, E. Chin, and D. Wagner, “Android Permissions: User Attention, Comprehension, and Behavior,” *Proc. SOUPS*, 2012.
- [12] OWASP, “OWASP API Security Top 10 – 2023 Edition.” <https://owasp.org/www-project-api-security/>
- [13] C. Mainka, V. Mladenov, J. Schwenk, and T. Wich, “SoK: Benchmarking the Effectiveness of OAuth Grant Type Enforcement Across Major SaaS Providers,” *Proc. ACM CCS*, 2022.
- [14] X. De Carne de Carnavalet and P. C. van Oorschot, “Third-Party Application Permission Practices: A Large-Scale Analysis of OAuth over Major Identity Providers,” *Proc. USENIX Security*, 2023.

- [15] T. Boucher, A. Kharraz, and E. Kirda, “Scope Guard: Static Policy Enforcement for SaaS Integration Permission Graphs,” *Proc. NDSS*, 2024.
- [16] E. Lundberg, M. Ekstedt, and R. Lagerström, “Runtime OAuth Scope Enforcement via Transparent API Proxy Middleware,” *Proc. IEEE EuroS&P*, 2024.
- [17] M. Zalewski, “American Fuzzy Lop (AFL),” 2014. <https://lcamtuf.coredump.cx/afl/>
- [18] J. Pereyda, “Boofuzz: A Fork and Fix of Sulley.” <https://github.com/jtpereyda/boofuzz>
- [19] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” *Proc. ICSE*, 2019.
- [20] A. Arcuri, “RESTful API Automated Test Case Generation with EvoMaster,” *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 1, 2019.
- [21] V. Atlidakis, P. Godefroid, and M. Polishchuk, “Checking Security Properties of Cloud Service REST APIs,” *Proc. IEEE S&P*, 2020.
- [22] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and Understanding Bugs in C Compilers,” *Proc. PLDI*, 2011.
- [23] T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana, “SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities,” *Proc. CCS*, 2017.
- [24] S. Kim, Y. Lee, and J. Choi, “DiffMicro: Differential Testing for Semantic Consistency in Microservice API Evolution,” *Proc. IEEE ICWS*, 2022.
- [25] F. Benamara, N. Mager, and M. Dacier, “DPIFuzz: A Differential Fuzzing Framework to Detect DPI Elusion Strategies for QUIC,” *Proc. ACSAC*, 2020.
- [26] A. Bulekov, B. Das, S. Hajnoczi, and M. Egele, “Fuzztruction: Using Fault Injection to Hack Fuzzing Mutators,” *Proc. USENIX Security*, 2023.
- [27] V. B. Livshits and M. S. Lam, “Finding Security Vulnerabilities in Java Applications with Static Analysis,” *Proc. USENIX Security*, 2005.
- [28] J. Clause, W. Li, and A. Orso, “Dytan: A Generic Dynamic Taint Analysis Framework,” *Proc. ISSTA*, 2007.
- [29] W. Enck et al., “TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones,” *ACM Trans. Comput. Syst.*, vol. 32, no. 2, 2014.

- [30] X. Pan, Y. Zhang, and M. Grace, “SynthAPI: Privacy-Preserving Synthetic Data Generation for API Security Testing,” *Proc. ACM CODASPY*, 2022.
- [31] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, Aug. 2020.
- [32] Y. Ding, R. Phan, and T. Pöppelmann, “Continuous Authorization for Third-Party SaaS Integrations under Zero Trust,” *Proc. IEEE CloudCom*, 2023.
- [33] D. Wermke et al., “A Large Scale Investigation of Obfuscation Use in Google Play,” *Proc. USENIX Security*, 2022.
- [34] R. Chandramouli, “SaaS Security Posture Management: Taxonomy, Challenges, and Research Directions,” NIST Internal Report 8467, 2023.
- [35] FIRST.Org, “Common Vulnerability Scoring System v3.1: Specification Document,” 2019. <https://www.first.org/cvss/specification-document>